WA3897-CAP Lab Guide

FedEx AI Native FDE Capstone

ascendient learning

Part of **Accenture**

Revision 1.0.0 published on 2026-05-29

Ascendient, LLC
1 California Street Suite 2900
San Francisco, CA 94111
https://www.ascendientlearning.com

USA: 1-877-517-6540, email: getinfousa@ascendientlearning.com
Canada: 1-877-812-8887 toll free, email: getinfo@ascendientlearning.com

Lab Materials and Environment Guidelines

Ascendient Learning is pleased to provide lab materials and environments as part of your learning experience. To ensure a safe and effective learning experience for everyone:

You may:

- Use the lab environment for your own coursework.
- Save any important files or data from the environment to your local computer or online storage (if allowed by your employer).
- Retain lab files for your personal use after the class ends.

You may not:

- Share your lab materials or environment with anyone outside of your course.
- Store any personal, private, confidential, or proprietary company information within the lab environment.
- Rely on the lab environment for data storage, as all work is permanently deleted after each class.
- Misuse the lab environment for any illegal, malicious, or non-class-related activities.

> ⓘ Important
>
> The lab environment is under Ascendient Learning's control and is not a part of your organization's IT ecosystem. Do not place your organization's information into the lab environment and always follow your organization's policies and procedures for working with IT systems.

Table of Contents

= Procurement and Vendor Intelligence Agent :lab-version: 1.0 :icons: font :source-highlighter: rouge :toc: left :toc-title: Contents :toclevels: 2 :imagesdir: images :experimental:

- ✓ Specify Pydantic v2 data models using spec-driven development with OpenSpec
- ✓ Implement four agent tool functions that enforce procurement policy rules
- ✓ Wire a Pydantic AI agent with structured output and a typed system prompt
- ✓ Write a pytest suite that covers all three decision outcomes — approve, deny, and escalate
- ✓ Complete a RAPID-compliant peer review and Go/No-Go decision gate

1. Introduction

1.1. Framework

This lab uses **spec-driven development** (SDD) throughout all five sessions. Spec-driven development separates the act of describing what a component must do from the act of writing the code that does it. You use the **OpenSpec** CLI to move through a consistent four-step workflow:

`openspec explore`	Read the domain — data shapes, edge cases, decision rules
`openspec propose`	Draft a specification in plain language
`openspec specify`	Formalize the spec into a machine-verifiable contract
`openspec verify`	Check whether the implementation satisfies the spec

GitHub Copilot in **Agent Mode** accelerates every phase. You will use it to generate proposals, draft implementations, write tests, and run a structured peer review against the FedEx RAPID Framework.

1.2. The Scenario

You are on FedEx's Enterprise Procurement Tools team. The procurement department processes several hundred purchase requests per week. Many routine requests consume analyst time that could be better spent on complex or high-value decisions.

Your team has been asked to build a **Procurement Intelligence Agent** that pre-screens purchase requests and produces structured recommendations — **approve**, **deny**, or **escalate** — with written rationale that a procurement officer can act on directly.

The agent is advisory. The procurement officer remains the final decision-maker. Your agent must be reliable, explainable, and consistent.

1.3. What You Will Build

Component	Description
`models.py`	Pydantic v2 `PurchaseRequest` and `ProcurementRecommendation` models
`data/loader.py`	Mock data loader — all file access goes through this module
`tools/`	Four tool functions: budget check, vendor duplication, policy compliance, risk assessment
`agent.py`	Pydantic AI agent that wires the tools together and produces recommendations
`tests/`	pytest suite covering all three decision outcomes
`openspec/`	Formal specifications for every component you build

Component	Description
`docs/`	RAPID compliance artifacts: peer review report and Go/No-Go checklist

1.4. Reference Data

The `mock_data/` directory is read-only. Review these files before Session 1:

File	Contents	Key facts
`budgets.json`	10 cost centers with budget allocations	CC-003 has $6,900 remaining
`vendors.json`	17 vendor records	V-006 is compliance-flagged; V-010 has an expired contract
`policies.json`	8 procurement policies	POL-004 prohibits catering at any amount
`requests.json`	15 sample purchase requests	`expected_outcome` field tells you what decision the agent should reach

2. Session 1: Explore the Domain and Draft Specifications

Duration: 3 hours
Copilot mode: Agent Mode (Plan agent for exploration, Edit mode for spec files)

2.1. Session 1 — Learning Objectives

By the end of Session 1, your group will:

- Understand the four checks the agent must perform
- Have OpenSpec specifications for `PurchaseRequest`, `ProcurementRecommendation`, and at least the budget-check tool
- Understand which sample requests produce each decision outcome and why

2.2. Step 1 — Set Up the Environment

1. Open VS Code in the `procurement-agent/` project directory.
2. Copy `.env.example` to `.env` and paste your Anthropic API key:

```
cp .env.example .env
# Edit .env and replace "your-api-key-here" with your key
```

3. Create a virtual environment and install dependencies:

```
uv venv .venv
source .venv/Scripts/activate
uv pip install -e ".[dev]"
```

4. Verify the install:

```
python -c "import pydantic_ai; print('OK')"
openspec --version
```

5. Open `user-stories.md` and read all five user stories as a group. Every commit you make in this capstone must include a user story ID in the format `[US-XXX] description`. Agree on your first commit message now — it will be the scaffold generated at the end of this session:
`[US-001] Initial project scaffold from OpenSpec specification`

6. Open `copilot-instructions.md` and read the **RAPID DevSecOps Conventions** section at the bottom. Confirm your group understands the commit format (ITC.014), the test capture command (ITC.003), and the backout plan requirement (ITC.013) before proceeding.

> ⓘ Note
>
> These three conventions mirror FedEx RAPID DevSecOps controls that would be enforced automatically in the enterprise CI/CD toolchain. In this capstone you practice them manually so the habits are in place when you exit training.

2.3. Step 2 — Read the Domain Data

1. Open `mock_data/requests.json` in VS Code. Read all 15 records. Note the `expected_outcome` field (`approve`, `deny`, `escalate`) and the `outcome_reason` that explains the decision.
2. Open `mock_data/policies.json`. For each policy, identify:
   - What condition triggers it
   - What decision it forces (deny vs. escalate)
   - Whether an amount threshold applies
3. Open `mock_data/vendors.json`. Find:
   - The vendor with `compliance_flag: true` (triggers escalation via POL-006)
   - The vendor with `contract_status: "expired"` (triggers denial via POL-005)
   - The two vendors that share the same `category: "office_supplies"` and both have active contracts (vendor duplication trigger)
4. Open `mock_data/budgets.json`. Find the cost centers where a sample request will exceed the remaining budget. Note which requests those are.

> ⓘ Note
>
> You are reading the mock data so you can write accurate specifications. The agent you build will access this data only through `data/loader.py`, not by reading the files directly.

2.4. Step 3 — Run OpenSpec Explore

1. Start a new Agent Mode session in VS Code by opening the Copilot chat panel and selecting **Agent** mode.
2. Paste the following prompt into the Agent Mode session:

```
@workspace Using openspec explore, analyze the procurement
domain in this project.
Read mock_data/requests.json, mock_data/policies.json,
mock_data/vendors.json,
and mock_data/budgets.json. Identify: (1) the data fields a
purchase request
contains, (2) the four checks the agent must perform, (3) the
decision rules that
map check results to approve/deny/escalate outcomes, and (4)
edge cases visible
in the sample data. Produce an exploration summary. Do not write
any code yet.
```

3. Review the exploration summary with your group. Correct any misunderstandings before moving to the proposal step.

> ⓘ Note
>
> **Pydantic AI — Structured Output**
>
> This capstone uses Pydantic AI's *structured output* feature. When you pass a Pydantic model as `result_type=` to the agent, the LLM is constrained to return data that matches that model's schema — the framework validates the response and raises an error if the shape is wrong.

```
from typing import Literal
from pydantic import BaseModel
from pydantic_ai import Agent

class ProcurementRecommendation(BaseModel):
    decision: Literal["approve", "deny", "escalate"]
    rationale: str

agent = Agent(
    "anthropic:claude-3-5-haiku-latest",
    output_type=ProcurementRecommendation,  # constrains LLM
output to this schema
)

result = agent.run_sync(prompt)
rec = result.data       # typed ProcurementRecommendation
instance
print(rec.decision)     # always "approve", "deny", or
"escalate"
print(rec.rationale)
# always a non-empty string (enforced by Pydantic)
```

The specification you write in the next step defines this contract. Whatever you specify for `ProcurementRecommendation` becomes the schema the agent is bound to at runtime.

2.5. Step 4 — Propose a Models Specification

1. In the Agent Mode session, run:

```
@workspace Using openspec propose, draft a specification for two
Pydantic v2
models: PurchaseRequest (input) and ProcurementRecommendation
(output).
The proposal must cover: field names, types, validation
constraints,
and the allowed values for the decision field.
Save the proposal to openspec/models-proposal.md.
```

2. Open `openspec/models-proposal.md` and review it with your group. Check that:
   - Every field in `mock_data/requests.json` is represented
   - `decision` is constrained to exactly `approve`, `deny`, `escalate`
   - `rationale` is required and non-empty
   - Appropriate Pydantic validators are suggested for numeric fields
3. Edit the proposal directly in VS Code to correct any gaps your group identifies.

2.6. Step 5 — Finalize the Models Spec

1. In the Agent Mode session, run:

```
@workspace Using openspec specify, convert openspec/models-
proposal.md into a
formal OpenSpec specification. Save the result to openspec/
models.spec.md.
The spec must include: field contracts, type constraints,
validation rules,
and examples for each of the three decision outcomes.
```

2. Verify the spec was created:

```
openspec list
```

3. You should see `models.spec.md` in the output.

> **Tip**
>
> A good spec is specific enough that two different developers would write nearly identical implementations from it. Vague language ("some kind of approval field") leads to inconsistent implementations.

2.7. Step 6 — Propose a Budget Check Tool Specification

1. The budget check tool is the simplest of the four tools. Specify it first.

2. In the Agent Mode session, run:

```
@workspace Using openspec propose, draft a specification for a
tool function
named check_budget. It must: accept a cost_center_id and
requested_amount,
look up the remaining budget for that cost center from
mock_data/budgets.json,
and return a structured result indicating whether the request is
within budget,
the remaining budget amount, and any overage amount. Save to
openspec/tool-budget-proposal.md.
```

3. Review and edit the proposal, then finalize it:

```
@workspace Using openspec specify, convert openspec/tool-budget-
proposal.md
into openspec/tool-budget.spec.md.
```

2.8. Session 1 Debrief

Before ending Session 1, confirm with your group:

- `openspec list` shows at least `models.spec.md` and `tool-budget.spec.md`
- Every member can explain the decision rule for each of the three outcomes
- You know which sample requests are designed to produce each outcome

---

3. Challenge 1: Specify All Four Tools

Complete this section if your group finishes the core steps early.

The agent requires four tool specifications, not just one. Draft and finalize specifications for the remaining three tools:

Tool: **`check_vendor_duplication`** — Given a vendor ID and purchase category, determine whether FedEx already has an active contract with another vendor in the same category. Return the list of conflicting vendor IDs and contract details. Reference POL-001 for the amount threshold above which this check triggers a deny.

Tool: `check_policy_compliance` — Given a purchase request, evaluate it against all eight policies in `mock_data/policies.json`. Return a list of violated policies, if any. Each violation must include the policy ID, the rule that was violated, and the forced decision (`deny` or `escalate`).

Tool: `assess_risk` — Given a vendor ID, return the vendor's risk profile: compliance flag status, contract status, and a computed risk level (`low`, `medium`, `high`, `critical`).

Save the finalized specs to:

- `openspec/tool-vendor-duplication.spec.md`
- `openspec/tool-policy-compliance.spec.md`
- `openspec/tool-risk-assessment.spec.md`

---

4. Challenge 2: Cross-Reference Specs Against Sample Data

Complete this section if your group finishes Challenge 1 early.

Manually trace each sample request in `mock_data/requests.json` through your four tool specifications. For each request, determine:

1. Which tools would be called
2. What each tool would return
3. Which decision rule applies
4. What rationale text the agent should produce

Build a traceability table and save it to `openspec/request-traceability.md`. This table will serve as the reference for writing test assertions in Session 4.

---

5. Session 2: Implement Models and the Data Loader

Duration: 3 hours
Copilot mode: Agent Mode (Edit mode for implementation)

5.1. Session 2 — Learning Objectives

By the end of Session 2, your group will:

- Have `models.py` implemented and verified against `openspec/models.spec.md`
- Have `data/loader.py` implemented with functions for each mock data file
- Have the budget check tool implemented and passing its spec

5.2. Step 1 — Implement `models.py`

1. Open the Copilot Agent Mode panel.
2. Run the following prompt to generate an initial implementation:

```
@workspace Implement models.py at the project root using the
specification in
openspec/models.spec.md. Use Pydantic v2 only. Follow the
conventions in
copilot-instructions.md. The file must define PurchaseRequest
and
ProcurementRecommendation. Do not add any code that is not
required by the spec.
```

3. Review the generated `models.py`:
   - Are all fields present with correct types?
   - Is `decision` typed as a `Literal["approve", "deny", "escalate"]`?
   - Is `rationale` validated as a non-empty string?
   - Are numeric fields validated with appropriate constraints (e.g., `gt=0`)?
   - Does every class have a docstring?
4. Make corrections in the file directly. Copilot Agent Mode will track changes.
5. Verify the implementation against the spec:

```
openspec verify openspec/models.spec.md
```

   Address any findings before continuing.

5.3. Step 2 — Implement `data/loader.py`

1. Create `data/__init__.py` (empty) so Python treats `data` as a package.
2. In the Agent Mode session, run:

```
@workspace Create data/loader.py. It must provide functions to
load each of the
four mock data files: load_budgets(), load_vendors(),
load_policies(), and
load_requests(). Each function must: read the appropriate file
from mock_data/,
parse the JSON, and return the data as a Python list. All file
paths must be
resolved relative to the project root, not hardcoded. Follow the
conventions
in copilot-instructions.md. Add a docstring to every function.
```

3. Test the loader manually:

```python
# In a Python REPL or a scratch script (do not commit)
from data.loader import load_vendors
vendors = load_vendors()
print(len(vendors))  # Expect 17
```

4. Verify that `mock_data/` is never accessed directly in any file other than `data/loader.py`. If Copilot placed file paths elsewhere, fix that now.

5.4. Step 3 — Specify and Implement the Budget Check Tool

1. If your group did not complete Challenge 1 in Session 1, draft and finalize `openspec/tool-budget.spec.md` now before writing any code.
2. Create `tools/__init__.py` (empty).
3. In the Agent Mode session, run:

```
@workspace Implement tools/budget.py using the specification in
openspec/tool-budget.spec.md. The function must be named
check_budget and
accept a cost_center_id (str) and requested_amount (float). Load
budget data
through data/loader.py — do not read mock_data/ directly. Add a
complete
docstring. Follow all conventions in copilot-instructions.md.
```

4. Review the generated tool:
   - Does it import from `data.loader`, not from `mock_data/`?
   - Does it have a docstring the agent could use to understand when to call it?
   - Does it handle the case where the cost center is not found?
   - Does it return a structured result (not a bare bool)?
5. Run OpenSpec verify:

```
openspec verify openspec/tool-budget.spec.md
```

5.5. Step 4 — Write a Smoke Test

1. Create `tests/__init__.py` (empty).
2. In the Agent Mode session, run:

```
@workspace Write a single pytest test in tests/test_budget.py
that imports
check_budget from tools.budget and verifies: (1) a request
within budget returns
a within-budget result, (2) a request exceeding the budget
returns an over-budget
result. Use cost center CC-003 with remaining budget $6,900 from
budgets.json.
Do not mock the data loader — use the real mock data files.
```

3. Run the test:

```
pytest tests/test_budget.py -v
```

4. Fix any failures before moving on.

5.6. Session 2 Debrief

Before ending Session 2, confirm:

- `openspec verify openspec/models.spec.md` passes
- `openspec verify openspec/tool-budget.spec.md` passes
- `pytest tests/test_budget.py` passes
- No file in `tools/` or `agent.py` reads from `mock_data/` directly

---

6. Challenge 1: Implement the Vendor Duplication Tool

Complete this section if your group finishes the core steps early.

1. If you did not write `openspec/tool-vendor-duplication.spec.md` in Session 1, draft and finalize it now.
2. Implement `tools/vendor_duplication.py` using the spec as your guide. The function must:
   - Accept a `vendor_id` (str) and `category` (str)
   - Load vendor data through `data/loader.py`
   - Return a result that includes the list of conflicting active vendor IDs
   - Apply the POL-001 threshold logic: the check only triggers for amounts exceeding $25,000 in a contracted category
3. Run `openspec verify openspec/tool-vendor-duplication.spec.md` and fix any findings.
4. Add a test in `tests/test_vendor_duplication.py` using REQ-008 as the test case (NovaPrint, office_supplies, $28,500 — should find V-001 and V-003 as conflicts).

---

7. Challenge 2: Implement the Policy Compliance Tool

Complete this section if your group finishes Challenge 1 early.

1. If you did not write `openspec/tool-policy-compliance.spec.md` in Session 1, draft and finalize it now.

2. Implement `tools/policy_compliance.py`. The function must evaluate a `PurchaseRequest` against all eight policies and return each violation with:
   - `policy_id`
   - `rule_description`
   - `forced_decision` (`deny` or `escalate`)
3. Cover the following policy cases in your tests:
   - POL-004 catering prohibition (REQ-009, $3,200 catering → deny)
   - POL-002 manager approval threshold (any request $10,000–$49,999)
   - POL-005 expired contract (REQ-007, Crestview Print → deny)

---

8. Session 3: Wire the Agent

Duration: 3 hours
Copilot mode: Agent Mode (Plan agent for architecture, Edit mode for wiring)

8.1. Session 3 — Learning Objectives

By the end of Session 3, your group will:

- Have all four tool functions implemented and OpenSpec-verified
- Have `agent.py` that wires the tools into a Pydantic AI agent
- Be able to run the agent manually against sample requests and see structured output

8.2. Step 1 — Implement Remaining Tools

1. If your group did not complete Challenges B and A in Session 2, implement the remaining tools now using the same pattern:
   - Write the OpenSpec spec first (`openspec specify`)
   - Implement the tool
   - Run `openspec verify`
2. The four tool files should be:
   - `tools/budget.py` — `check_budget`

- `tools/vendor_duplication.py` — `check_vendor_duplication`
- `tools/policy_compliance.py` — `check_policy_compliance`
- `tools/risk_assessment.py` — `assess_risk`

3. Verify all tool specs:

```
openspec verify-all
```

8.3. Step 2 — Design the Agent Architecture

1. Open a fresh Agent Mode session.
2. Run the following planning prompt before writing any agent code:

```
@workspace I need to design the architecture for agent.py in
this Pydantic AI
project. The agent must: accept a PurchaseRequest, call all four
tools defined
in tools/, produce a ProcurementRecommendation with decision
always one of
approve/deny/escalate and a non-empty rationale. Plan the agent
definition —
what system prompt it needs, how tools should be registered, how
errors should
be handled. Do not write any code yet. Output the plan as a
numbered list.
```

3. Review the plan with your group. The plan should address:
   - Where the decision logic lives (system prompt vs. tool post-processing)
   - How conflicting tool outputs are resolved (e.g., one tool says deny, another escalate)
   - How tool errors are surfaced without crashing the recommendation

8.4. Step 3 — Write an Agent Specification

Before implementing `agent.py`, write its OpenSpec:

```
@workspace Using openspec specify, write a formal specification for
agent.py.
The spec must describe: the agent's inputs and outputs (using
models.py types),
the four tools it uses, the decision priority order when multiple
checks fire,
error handling behavior, and the system prompt constraints.
Save to openspec/agent.spec.md.
```

8.5. Step 4 — Implement `agent.py`

1. In the Agent Mode session, run:

   ```
   @workspace Implement agent.py at the project root using the
   specification in
   openspec/agent.spec.md. Use pydantic_ai.Agent with the Anthropic
   model.
   Register all four tools from the tools/ directory. The system
   prompt must
   enforce the decision priority: escalate > deny > approve.
   The model must read the ANTHROPIC_API_KEY from the environment
   using python-dotenv.
   Follow all conventions in copilot-instructions.md.
   ```

2. Review the generated `agent.py`:
   - Is `Agent` imported from `pydantic_ai`?
   - Is the output type set to `ProcurementRecommendation`?
   - Are all four tools registered?
   - Does the system prompt explicitly state the decision priority order?
   - Is `load_dotenv()` called before the agent is instantiated?

3. Fix any issues identified in the review.

8.6. Step 5 — Test the Agent Manually

1. Write a quick test script (do not commit this file):

```
# scratch_test.py — delete before Session 4
import asyncio
from agent import agent
from models import PurchaseRequest

# REQ-001: straightforward approve
req = PurchaseRequest(
    request_id="REQ-001",
    requester="j.smith@fedex.com",
    cost_center_id="CC-001",
    vendor_id="V-002",
    category="office_supplies",
    description="Standard office paper and toner",
    amount=1250.00,
)

async def main():
    result = await agent.run(str(req))
    print(result.data)

asyncio.run(main())
```

2. Run the script and verify the agent returns an `approve` recommendation with a non-empty rationale.
3. Test a deny case using REQ-009 (catering, POL-004).
4. Test two escalate cases to verify both escalation paths:
    - **REQ-011** (Vertex Consulting, compliance flag) — escalation driven by a risk tool finding
    - **REQ-014** (Orion Data Systems, $47,500 hardware — within 5% of the $50,000 director approval threshold) — escalation driven by near-threshold amount logic

> **Tip**
>
> If the agent produces the wrong decision for REQ-014, the system prompt likely lacks explicit near-threshold logic. Add: "If the request amount is within 5% of the director approval threshold, escalate." The most common issue overall is ambiguous priority language — "should prefer escalate" is weaker than "if any tool returns escalate, the final decision is always escalate."

8.7. Step 6 — Verify the Agent Spec

```
openspec verify openspec/agent.spec.md
```

Address all findings before Session 4.

8.8. Session 3 Debrief

Before ending Session 3, confirm:

- `openspec verify-all` passes (no failing specs)
- The agent returns correct decisions for REQ-001 (approve), REQ-009 (deny), REQ-011 (escalate — compliance flag), and REQ-014 (escalate — near-threshold amount)
- Tool errors are caught — test by passing an invalid cost center ID and confirming the agent returns a non-empty rationale rather than an exception

---

9. Challenge 1: Harden Error Handling

Complete this section if your group finishes the core steps early.

Tool failures must not produce silent errors. Implement a consistent error handling pattern:

1. Each tool should catch `FileNotFoundError`, `KeyError`, and general exceptions separately, and return a typed error result rather than raising.
2. The agent system prompt must instruct the model that if a tool returns an error, it must reference the error in the rationale and escalate the request.
3. Write a test that patches `data.loader.load_budgets` to raise a `FileNotFoundError` and confirms the agent returns an `escalate` recommendation with a rationale mentioning the data loading failure.

---

10. Challenge 2: System Prompt Precision

Complete this section if your group finishes Challenge 1 early.

The rationale field is the most important part of the recommendation — it is what the procurement officer reads before making a final decision.

1. Define a rationale template in the system prompt. The rationale must:
    - Name the specific check(s) that drove the decision
    - Include relevant amounts, vendor names, or policy IDs
    - Use complete sentences (no bullet points)
    - Be 2–4 sentences in length
2. Run all 15 sample requests and evaluate each rationale against the template. Record cases where the rationale is vague or missing key context.
3. Refine the system prompt until all 15 rationales meet the template criteria.

---

11. Session 4: Test and Peer Review

Duration: 3 hours
Copilot mode: Agent Mode (plan review skill, then generate test suite)

11.1. Session 4 — Learning Objectives

By the end of Session 4, your group will:

- Have a pytest suite that covers all four required cases
- Have run the RAPID peer review Agent Skill and received a written findings document
- Have addressed all peer review findings or documented why they are acceptable

11.2. Step 1 — Write the Core Test Suite

1. Open a fresh Agent Mode session.
2. Run the following prompt:

```
@workspace Write a pytest test suite in tests/test_agent.py. The
suite must cover
four cases using the sample requests in mock_data/requests.json:
(1) approve — use REQ-001
(2) deny — use REQ-006 (budget overage on CC-003)
(3) policy-deny — use REQ-009 (catering prohibition POL-004)
(4) escalate — use REQ-011 (compliance-flagged vendor Vertex
Consulting)
Each test must assert that result.data.decision equals the
expected outcome and
that result.data.rationale is a non-empty string. Use pytest-
asyncio.
```

3. Review the generated tests:
   - Do all four tests import `PurchaseRequest` from `models`?
   - Are the request fields accurate (match the JSON records)?
   - Is `@pytest.mark.asyncio` or `asyncio_mode = "auto"` handling async correctly?
4. Run the full suite:

```
pytest tests/ -v
```

5. Fix all failures before continuing.

> ⓘ Important
>
> Do not delete or modify tests to make them pass. Fix the underlying implementation or data — not the tests.

11.3. Step 2 — Run the RAPID Peer Review

1. Start a new Agent Mode session in VS Code.
2. Invoke the peer review Agent Skill by typing the following in the Copilot chat panel:

```
@workspace Run the peer review Agent Skill defined in
.github/skills/rapid-peer-review.md. Perform a full RAPID ITC.
009 code review
of the current implementation. Evaluate all six criteria and
save the findings
to docs/rapid-peer-review.md.
```

3. Wait for Copilot to complete the review. It will:
   - Identify modified files using `git diff`
   - Evaluate six criteria (file inventory, author/reviewer separation, InfoSec, architecture, documentation, behavioral scope)
   - Write `docs/rapid-peer-review.md` with ratings and findings
4. Open `docs/rapid-peer-review.md` and read it with your group.

11.4. Step 3 — Address Peer Review Findings

1. For every criterion rated **Needs Attention** or **Fail**:
   i. Identify the specific file or pattern that caused the finding
   ii. Fix the issue in the implementation
   iii. Commit the fix
   iv. Note the resolution in the Required Actions section of the peer review document
2. Re-run `pytest tests/ -v` after each fix to confirm nothing regressed.
3. When all findings are addressed (or formally accepted with a documented rationale), the peer review is complete.

> ⓘ Note
>
> "Formally accepted" means your group writes one sentence in the peer review document explaining why the finding is acceptable for this training context. For example, an Author/Reviewer Separation finding of "Needs Attention" is expected when a single developer is doing both — document this explicitly.

11.5. Step 4 — Full OpenSpec Verify

1. Run `openspec verify-all` one final time to confirm all specs pass:
   ```
   openspec verify-all
   ```
2. Resolve any remaining findings.

11.6. Step 5 — Capture Test Results and Complete RAPID Compliance Artifacts

1. Run the full test suite with results capture (ITC.003):
   ```
   pytest tests/ -v --tb=short --junitxml=docs/test-results.xml
   ```
   `docs/test-results.xml` is a machine-readable record of all test outcomes. All tests must pass before you commit this file.
2. Open `backoutPlan.md` in VS Code. Fill in the required fields (ITC.013):
   - **Section 1** — run `git log --oneline -1` and paste the hash into "Last stable commit hash"
   - **Section 5** — add your group members' names and contact information
3. Commit all three compliance artifacts together:
   ```
   git add docs/test-results.xml docs/go-no-go-checklist.md
   backoutPlan.md
   git commit -m "[US-005] Add ITC.003 test results and ITC.013
   backout plan"
   ```

11.7. Session 4 Debrief

Before ending Session 4, confirm:

`pytest tests/ -v --tb=short --junitxml=docs/test-results.xml` — all tests pass; `docs/test-results.xml` committed
`docs/rapid-peer-review.md` exists and has ratings for all six criteria
All **Fail** findings are resolved

`openspec verify-all` passes
`backoutPlan.md` has stable baseline commit hash and contacts filled in

---

12. Challenge 1: Parametrized Test Coverage

Complete this section if your group finishes the core steps early.

Extend the test suite using `@pytest.mark.parametrize` to cover more sample requests. At minimum, add parametrized cases for:

- All four deny-type requests (REQ-006, REQ-007, REQ-008, REQ-009)
- Both escalate-type requests that are clearly deterministic (REQ-010, REQ-011)
- Three approve-type requests (REQ-001, REQ-002, REQ-003)

Structure the test so that each parametrized case receives a `request_id`, loads the corresponding record from `mock_data/requests.json`, runs the agent, and asserts the `expected_outcome` field matches `result.data.decision`.

---

13. Challenge 2: Test Error Paths

Complete this section if your group finishes Challenge 1 early.

The acceptance criteria state: "Tool errors are caught and reflected in the output." Write two tests in `tests/test_error_handling.py` that verify this behavior:

Test 1: Patch `data.loader.load_budgets` to raise a `RuntimeError`. Confirm the agent returns a recommendation (not an unhandled exception) with a non-empty rationale mentioning the failure.

Test 2: Pass a `PurchaseRequest` with a `vendor_id` that does not exist in `mock_data/vendors.json`. Confirm the agent escalates with a rationale mentioning the unknown vendor.

Use `unittest.mock.patch` for Test 1.

---

14. Session 5: Integration, Go/No-Go, and Showcase

Duration: 3 hours
Copilot mode: Agent Mode for final fixes only

14.1. Session 5 — Learning Objectives

By the end of Session 5, your group will:

- Have run the agent against all 15 sample requests and verified all three decision types are reachable
- Have a completed `docs/go-no-go-checklist.md`
- Have presented your agent and key design decisions to the group

14.2. Step 1 — Full Integration Run

1. Write a script `run_all_requests.py` at the project root:

```
"""Run the agent against all 15 sample requests and compare to
expected outcomes."""
import asyncio
import json
from pathlib import Path
from agent import agent
from models import PurchaseRequest


async def main() -> None:
    requests_data = json.loads(
        Path("mock_data/
requests.json").read_text(encoding="utf-8")
    )

    results = {"approve": 0, "deny": 0, "escalate": 0,
"mismatch": 0}

    for req_data in requests_data:
        expected = req_data["expected_outcome"]
        request = PurchaseRequest(**{
            k: v for k, v in req_data.items()
            if k not in {"expected_outcome", "outcome_reason"}
        })
        result = await agent.run(str(request))
        decision = result.data.decision
        match = "✓" if decision == expected else "✗"
        results[decision] += 1
        if decision != expected:
            results["mismatch"] += 1
        print(f"{match} {req_data['request_id']}:
expected={expected}, got={decision}")
        print(f"  Rationale: {result.data.rationale[:80]}...")
        print()

    print(f"\nSummary: approve={results['approve']}
deny={results['deny']} "
          f"escalate={results['escalate']}
mismatches={results['mismatch']}")


asyncio.run(main())
```

2. Run the script:
   ```
   python run_all_requests.py
   ```
3. Count the results. You must see at least one `approve`, one `deny`, and one `escalate` in the output to satisfy the acceptance criteria.
4. For any mismatches (expected ≠ actual), investigate whether the issue is in:
   - The system prompt (priority order)
   - A tool returning an incorrect result
   - The policy data being misread
5. Fix mismatches using Agent Mode and re-run until the count of mismatches is zero for the deterministic requests (REQ-001–REQ-014). REQ-015 is intentionally ambiguous — document what your agent produces and why.

14.3. Step 2 — Final Acceptance Criteria Check

Work through every acceptance criterion in `README.md` and confirm it is met:

```
# Decision types reachable
python run_all_requests.py

# All four tests pass
pytest tests/ -v

# OpenSpec suite passes
openspec verify-all
```

Confirm `docs/rapid-peer-review.md` exists and all findings are addressed.

14.4. Step 3 — Complete the Go/No-Go Checklist

1. Open `docs/go-no-go-checklist.md`.
2. Fill in every field:
   - Date: today's date
   - Release description: what your agent does in one sentence
   - Test results: paste the pytest summary line
   - Paste the `openspec verify-all` output

    - Peer review rating from `docs/rapid-peer-review.md`
    - Outstanding defects: note REQ-015 behavior if your outcome differs from expected

3. Write the Decision Rationale — minimum two sentences explaining your Go decision. Reference the test results, peer review rating, and acceptance criteria status.
4. Mark the decision checkbox (Go, No-Go, or Conditional Go).

14.5. Step 4 — Group Showcase

Each group has **5 minutes** to present. Structure your showcase as follows:

Time	Content
0:00–1:00	Demo: run REQ-011 (escalate path) live and show the recommendation
1:00–2:30	Walk the spec: open one OpenSpec file and explain what it specifies and why
2:30–4:00	Show the peer review: open `docs/rapid-peer-review.md` and explain one finding and how you addressed it
4:00–5:00	One design decision you would make differently if you were building this for production

14.6. Session 5 Debrief

After all groups present, the instructor will facilitate a discussion:

- Which check was most difficult to implement correctly, and why?
- What would change about the agent design if the procurement officer needed to override a recommendation?
- How would you version the agent if the policy list changes?

---

15. Challenge 1: Handle the Ambiguous Case

Complete this section if your group finishes the core steps early.

REQ-015 involves FastTrack Couriers, category `courier_services`, amount $4,100 on cost center CC-005 (remaining budget $5,500). No policy explicitly prohibits this request, and the vendor has no compliance issues or expired contracts.

The `expected_outcome` field for REQ-015 is `ambiguous`.

1. Run REQ-015 through your agent and record the decision and rationale.
2. Modify the system prompt so the agent explicitly addresses tight-budget scenarios (remaining budget < 20% after the purchase) by escalating with a rationale that flags the low remaining budget.
3. After the modification, REQ-015 should consistently produce `escalate`. Write a test that asserts this.
4. Consider: is this the right behavior? Write a two-paragraph justification in `docs/req-015-decision.md` explaining your choice.

---

16. Challenge 2: Add a Confidence Score

Complete this section if your group finishes Challenge 1 early.

Extend `ProcurementRecommendation` with a `confidence` field:

1. Add `confidence: float` — a value between 0.0 and 1.0 inclusive.
2. Update `openspec/models.spec.md` with the new field contract.
3. Update the agent system prompt to instruct the model:
    - 1.0: only one check fired and its outcome is unambiguous (e.g., sole catering prohibition)
    - 0.8–0.9: two or more checks fired and all agree
    - 0.5–0.7: checks fired but at least one is borderline (near a threshold)
    - Below 0.5: the agent could not determine a clear decision; escalate
4. Run `openspec verify openspec/models.spec.md` to confirm the spec is updated.
5. Add a test asserting that REQ-009 (catering, single unambiguous deny) produces `confidence >= 0.9`.

17. Conclusion

You built a production-pattern procurement intelligence agent from specification through verified implementation:

- Specified `PurchaseRequest` and `ProcurementRecommendation` as formal OpenSpec contracts, defining the input and output schemas before writing a line of implementation code
- Implemented four policy-aware tool functions — budget check, vendor duplication, policy compliance, and risk assessment — each verified against live mock data
- Wired a Pydantic AI agent with a structured system prompt and typed output that consistently produces `approve`, `deny`, or `escalate` with auditable rationale
- Validated all three decision outcomes with a pytest suite and captured results as a RAPID ITC.003-compliant test execution record
- Completed a structured peer review using the RAPID Agent Skill and a Go/No-Go decision gate with written rationale

The spec-driven workflow you practiced here — explore, propose, specify, implement, verify — applies to any agent component, not just procurement. The pattern scales to multi-agent systems, live data sources, and enterprise policy engines.

17.1. Next Steps

- Extend `ProcurementRecommendation` with a `confidence_score` field and update the system prompt to populate it — then add an assertion in your test suite
- Replace the `mock_data/` JSON files with a database loader and measure the latency impact on agent response time
- Add a second agent that drafts a reply email to the requestor based on the recommendation, chaining the two agents in sequence
- Explore Pydantic AI's streaming API to return the rationale incrementally for large or complex purchase requests

18. Appendix A — Quick Reference

18.1. OpenSpec Commands

```
openspec explore          # Analyze domain; summarize data shapes
and rules
openspec propose          # Draft a spec in plain language
openspec specify          # Formalize a proposal into a machine-
verifiable spec
openspec verify <file>    # Check one spec against the
implementation
openspec verify-all       # Check all specs in the openspec/
directory
openspec list             # List all specs and their status
```

18.2. Decision Priority Order

Priority	Decision	When
1 (highest)	`escalate`	Any tool returns escalate, compliance flag, director threshold, budget overage + near threshold
2	`deny`	Policy prohibition, expired contract, budget overage (clear), vendor duplication above threshold
3 (lowest)	`approve`	All checks pass; no flags, no policy violations, within budget

18.3. Tool Summary

Tool	Key inputs	Key output
`check_budget`		within_budget bool, remaining, overage

Tool	Key inputs	Key output
	`cost_center_id`, `requested_amount`	
`check_vendor_duplication`	`vendor_id`, `category`, `amount`	conflicting_vendors list
`check_policy_compliance`	`PurchaseRequest`	list of violated policies with forced decisions
`assess_risk`	`vendor_id`	compliance_flag, contract_status, risk_level

18.4. Acceptance Criteria Checklist

#	Criterion
1	Agent accepts `PurchaseRequest`, returns `ProcurementRecommendation`
2	Decision is always `approve`, `deny`, or `escalate`
3	Every recommendation includes a non-empty `rationale`
4	All four checks performed: budget, vendor duplication, policy, risk
5	Tool errors are caught and reflected in output
6	All three decision types are reachable with sample requests
7	pytest suite passes: approve, deny, policy-deny, escalate cases
8	`openspec verify-all` passes
9	`docs/rapid-peer-review.md` exists and all findings addressed

#	Criterion
10	`docs/go-no-go-checklist.md` is completed with written decision rationale

= Release Readiness and Dependency Risk Agent :lab-version: 1.0 :icons: font :source-highlighter: rouge :toc: left :toc-title: Contents :toclevels: 2 :imagesdir: images :experimental:

- ✓ Read and understand a brownfield release pipeline before writing a single line of new code
- ✓ Specify missing components using OpenSpec within an existing architecture
- ✓ Implement two pipeline stubs — dependency scanner and ticket client — to contract
- ✓ Wire a Pydantic AI agent with structured output that integrates with the existing pipeline
- ✓ Write a pytest suite that covers all three decision outcomes — release, hold, and escalate
- ✓ Complete a RAPID-compliant peer review and Go/No-Go decision gate

19. Introduction

19.1. Framework

This lab uses **spec-driven development** (SDD) in a **brownfield** setting — you are joining a codebase that already exists. Your first task is to understand what is there before adding anything new.

The OpenSpec workflow applies throughout all five sessions:

`openspec explore`	Understand the existing system; identify gaps
`openspec propose`	Draft specifications for the components you will add
`openspec specify`	Formalize proposals into machine-verifiable contracts
`openspec verify`	Confirm implementations match their specs

A pre-authored spec (`openspec/pipeline-architecture.md`) documents the existing pipeline. Your group will read it in Session 1 and use it as the boundary line between "what already exists" and "what you must build."

GitHub Copilot in **Agent Mode** accelerates every phase. You will use it to explore the brownfield codebase, implement stub functions, wire the agent, write tests, and run a RAPID peer review.

19.2. The Scenario

You are joining the FedEx Shipping Platform team mid-sprint. The team runs a release pipeline that has been in production for several months. The existing deploy gate (`pipeline/deploy.py`) checks only CI pass/fail.

Two recent incidents prompted leadership to expand the gate:

1. A deployment shipped with a HIGH-severity CVE in a transitive dependency that was caught only after a post-deploy security scan.
2. A major version bump landed in production with breaking API changes that were never documented in the changelog — downstream teams were not notified.

Your task: integrate a **Release Readiness Agent** into the pipeline. The agent scans dependencies for known vulnerabilities, checks for open blocking tickets, and validates changelog coherence. It produces a structured recommendation — **release**, **hold**, or **escalate** — with written rationale that a release manager can act on before the existing deploy gate runs.

The agent is advisory. The deploy gate and the human release manager remain the final decision-makers.

19.3. The Brownfield Pipeline

Before writing any code, understand what already exists:

File	Status	Description
`pipeline/runner.py`	Functional	Orchestrates a pipeline run; evaluates CI results via the deploy gate
`pipeline/deploy.py`	Functional	Deploy gate — approves builds that pass all CI checks
`pipeline/changelog_reader.py`	Functional	Parses CHANGELOG.md into structured version entries
`pipeline/dependency_scanner.py`	Stub	Scans requirements files against CVE database — NOT implemented
`pipeline/ticket_client.py`	Stub	Queries open/blocking tickets for a component — NOT implemented

You will implement the two stubs and build the agent that coordinates all four checks. You must not break the three functional modules.

19.4. Reference Data

The `mock_data/` directory is read-only:

File	Contents	Key facts
`cve_database.json`	12 CVE records for common	CVE-2024-5678 is CRITICAL (CVSS 9.1)

File	Contents	Key facts
	Python packages	
`tickets.json`	15 tickets across 6 components	LABEL-203 and SHIP-1042 are blocking
`ci_results.json`	8 CI run records	CI-2024-005 and CI-2024-006 have test failures
`release_requests.json`	9 sample release requests	`expected_outcome` field shows the target decision
`requirements_shipping_api.txt`	Shipping API deps	cryptography 41.0.5 → CRITICAL CVE match
`requirements_label_service.txt`	Label service deps	requests 2.30.0 → HIGH CVE match
`requirements_notification_service.txt`	Notification service deps	All packages at safe versions
`CHANGELOG.md`	Multi-release changelog	v3.0.0 entry has a major-version coherence gap

20. Session 1: Read the Brownfield Codebase and Draft Missing Specs

Duration: 3 hours
Copilot mode: Agent Mode (Plan agent for exploration)

20.1. Session 1 — Learning Objectives

By the end of Session 1, your group will:

- Understand every functional pipeline module's purpose and interface
- Understand the stub contracts defined in `openspec/pipeline-architecture.md`
- Have drafted OpenSpec specifications for all components you must build
- Know which sample requests produce each decision outcome and why

20.2. Step 1 — Set Up the Environment

1. Open VS Code in the `release-agent/` project directory.
2. Copy `.env.example` to `.env` and paste your Anthropic API key:

```
cp .env.example .env
# Edit .env and replace "your-api-key-here" with your key
```

3. Create a virtual environment and install dependencies:

```
uv venv .venv
source .venv/Scripts/activate
uv pip install -e ".[dev]"
```

4. Open `user-stories.md` and read all five user stories as a group. Every commit you make in this capstone must include a user story ID in the format `[US-XXX] description`. Agree on your first commit message now — it will be the integration spec generated at the end of this session:
`[US-005] Initial integration spec from OpenSpec analysis`

5. Open `copilot-instructions.md` and read the **RAPID DevSecOps Conventions** section at the bottom. Confirm your group understands the commit format (ITC.014), the test capture command (ITC.003), and the backout plan requirement (ITC.013) before proceeding.

> ⓘ Note
>
> These three conventions mirror FedEx RAPID DevSecOps controls that would be enforced automatically in the enterprise CI/CD toolchain. In this capstone you practice them manually so the habits are in place when you exit training.

1. Verify the install and run the existing pipeline to see its current behavior:

```
python -c "import pydantic_ai; print('OK')"
openspec --version
python -m pipeline.runner
```

2. Read the output of `python -m pipeline.runner`. Note which release requests the existing deploy gate approves and why.

20.3. Step 2 — Read the Functional Pipeline Modules

1. Open `pipeline/changelog_reader.py`. Read the entire file. Answer these questions with your group:
   - What does `detect_major_version_gap()` return when it finds a gap?
   - How does it detect that a major version bump has no breaking-change documentation?
   - What happens when the changelog does not contain an entry for the version you query?
2. Open `pipeline/deploy.py`. Read the `DeployGate.evaluate()` method. What are the four conditions that cause a denial?
3. Open `pipeline/runner.py`. Trace the execution path of `PipelineRunner.run()` for a request that has a valid `ci_run_id`. What does the runner return when the CI result file is missing?

> ⓘ Note
>
> You must not modify these three files during this lab. If you find a bug, note it in `docs/rapid-peer-review.md` during Session 4 — do not fix it inline.

20.4. Step 3 — Read the Pre-Authored Spec

1. Open `openspec/pipeline-architecture.md`. This document describes the existing pipeline and the stub interfaces.
2. Find the section "What Is Not Described Here." This section lists exactly what your group must specify and implement.
3. Run:

   ```
   openspec list
   ```

   You should see `pipeline-architecture.md` listed. The remaining specs you will create in this session do not exist yet.

20.5. Step 4 — Read the Sample Release Requests

1. Open `mock_data/release_requests.json`. Read all nine records.
2. For each record, trace the decision manually:
   - Look up the requirements file — check which CVEs match using `cve_database.json`
   - Look up the component and version — find all blocking tickets in `tickets.json`
   - Look up the CI run — check the result in `ci_results.json`
   - Check the changelog version entry in `mock_data/CHANGELOG.md`
3. Record each request's expected outcome and the checks that drove it in a scratch document. You will use this as the basis for your test cases in Session 4.

> **Tip**
>
> The `expected_outcome` field tells you the correct answer. The `outcome_reason` tells you which check is responsible. Both fields are removed when the agent processes real requests — the agent must derive the same conclusion from the data.

20.6. Step 5 — OpenSpec Explore

1. Open a new Agent Mode session.
2. Run the following prompt:

```
@workspace Using openspec explore, analyze the release readiness
domain in this
project. Read mock_data/cve_database.json, mock_data/
tickets.json,
mock_data/ci_results.json, mock_data/release_requests.json, and
mock_data/CHANGELOG.md. Also read openspec/pipeline-
architecture.md to understand
what already exists. Identify: (1) the four checks the agent
must perform,
(2) the decision rules mapping check results to release/hold/
escalate outcomes,
(3) what is already implemented versus what must be built.
Produce an exploration
summary. Do not write any code yet.
```

3. Review the summary with your group. Verify it matches the decision rules table in `copilot-instructions.md`.

> ⓘ Note
>
> **Pydantic AI — Structured Output**
>
> This capstone uses Pydantic AI's *structured output* feature. When you pass a Pydantic model as `result_type=` to the agent, the LLM is constrained to return data that matches that model's schema — the framework validates the response and raises an error if the shape is wrong.

```
from typing import Literal
from pydantic import BaseModel
from pydantic_ai import Agent

class ReleaseRecommendation(BaseModel):
    decision: Literal["release", "hold", "escalate"]
    rationale: str

agent = Agent(
    "anthropic:claude-3-5-haiku-latest",
    output_type=ReleaseRecommendation,  # constrains LLM output
to this schema
)

result = agent.run_sync(prompt)
rec = result.data       # typed ReleaseRecommendation instance
print(rec.decision)     # always "release", "hold", or
"escalate"
print(rec.rationale)
# always a non-empty string (enforced by Pydantic)
```

The specification you write in the next step defines this contract. Whatever you specify for `ReleaseRecommendation` becomes the schema the agent is bound to at runtime.

20.7. Step 6 — Propose and Specify Missing Components

1. The components you must specify (from `openspec/pipeline-architecture.md`) are:
   - `ReleaseRequest` and `ReleaseRecommendation` models
   - `data/loader.py`
   - Implementations for `dependency_scanner.scan_dependencies` and `is_version_affected`
   - Implementations for `ticket_client.get_blocking_tickets` and `get_open_tickets`
   - `agent.py`
2. Use OpenSpec to draft and finalize specs for the models first:

```
@workspace Using openspec propose, draft a specification for two
Pydantic v2 models:
ReleaseRequest (matching the fields in mock_data/
release_requests.json) and
ReleaseRecommendation (with decision as one of release/hold/
escalate and a required
non-empty rationale). Save the proposal to openspec/models-
proposal.md.
```

3. Review and finalize:

```
@workspace Using openspec specify, convert openspec/models-
proposal.md into
openspec/models.spec.md. Include field contracts, validation
constraints, and
one example for each of the three decision outcomes.
```

20.8. Step 7 — Specify the Stub Implementations

1. Specify the dependency scanner implementation (the function bodies, not just the stubs already defined in `pipeline/dependency_scanner.py`):

```
@workspace Using openspec specify, write a spec for the
implementation of
pipeline/dependency_scanner.py. The spec must cover: how
requirements files are
parsed, how version comparison works using
packaging.version.Version, what the
return value looks like for a matched CVE, and how errors are
propagated.
Save to openspec/dependency-scanner.spec.md.
```

2. Specify the ticket client implementation:

```
@workspace Using openspec specify, write a spec for the
implementation of
pipeline/ticket_client.py. Cover: how component matching works
(case-insensitive),
how version wildcard patterns like "2.4.x" are matched, and
return value structure.
Save to openspec/ticket-client.spec.md.
```

20.9. Session 1 Debrief

Before ending Session 1, confirm:

- `openspec list` shows `models.spec.md`, `dependency-scanner.spec.md`, and `ticket-client.spec.md`
- Every member can trace REL-001 (release), REL-003 (hold), and REL-006 (escalate) through the decision rules
- Your group agrees on which components are "brownfield" (do not touch) vs. "yours to build"

---

21. Challenge 1: Specify the Data Loader and Agent

Complete this section if your group finishes the core steps early.

Draft and finalize specs for the two remaining components:

`data/loader.py`: Functions `load_cve_database()`, `load_tickets()`, `load_ci_results()`, and `load_release_requests()`. Each reads the corresponding `mock_data/` file and returns a Python list. Paths are resolved from the project root.

`agent.py`: The Pydantic AI agent that accepts a `ReleaseRequest`, calls four tools (CVE scan, ticket query, changelog gap detection, CI gate), and returns a `ReleaseRecommendation`. Specify the decision priority order in the system prompt contract.

Save to `openspec/data-loader.spec.md` and `openspec/agent.spec.md`.

---

22. Challenge 2: Version Matching Deep Dive

Complete this section if your group finishes Challenge 1 early.

The ticket client must match version wildcard patterns like `"2.4.x"` against release versions like `"2.4.1"`. Write a detailed section in your `openspec/ticket-client.spec.md` covering:

- Exact match: `"3.0.0"` matches only `"3.0.0"`
- Major wildcard: `"3.x"` matches `"3.0.0"`, `"3.1.2"`, `"3.99.0"` but not `"4.0.0"`
- Minor wildcard: `"2.4.x"` matches `"2.4.0"` and `"2.4.99"` but not `"2.5.0"`
- Edge cases: what if `affected_versions` is empty? What if the version string is malformed?

Extend the spec with a table of 10 test cases: input version, pattern, expected match result.

---

23. Session 2: Implement the Stubs

Duration: 3 hours
Copilot mode: Agent Mode (Edit mode for implementation)

23.1. Session 2 — Learning Objectives

By the end of Session 2, your group will:

- Have `pipeline/dependency_scanner.py` fully implemented
- Have `pipeline/ticket_client.py` fully implemented
- Have `data/loader.py` implemented
- Both stub implementations verified against their OpenSpec specs

23.2. Step 1 — Implement `data/loader.py`

1. Create `data/__init__.py` (empty) so Python treats `data` as a package.
2. In Agent Mode, run:

```
@workspace Implement data/loader.py using the specification in
openspec/data-loader.spec.md (or create that spec now if it does
not exist).
Provide four functions: load_cve_database(), load_tickets(),
load_ci_results(),
and load_release_requests(). All paths must be resolved relative
to the project
root. Each function must have a complete docstring. Follow
copilot-instructions.md.
```

3. Test manually:

```
from data.loader import load_cve_database
cves = load_cve_database()
print(len(cves))  # Expect 12
print(cves[0]["cve_id"])  # Expect a CVE ID string
```

23.3. Step 2 — Implement `is_version_affected`

1. The `is_version_affected` helper in `pipeline/dependency_scanner.py` must be implemented first because `scan_dependencies` depends on it.
2. In Agent Mode, run:

```
@workspace Implement is_version_affected() in pipeline/
dependency_scanner.py.
It must use packaging.version.Version for comparison — not
string comparison.
Remove the NotImplementedError. Keep the existing docstring and
examples unchanged.
Do not modify scan_dependencies() in this step.
```

3. Test the function in isolation:

```
from pipeline.dependency_scanner import is_version_affected
assert is_version_affected("41.0.5", "41.0.6") is True
assert is_version_affected("41.0.6", "41.0.6") is False
assert is_version_affected("2.0.0", "1.99.0") is False
print("All assertions pass")
```

4. Run `openspec verify openspec/dependency-scanner.spec.md` and confirm the partial implementation satisfies the helper-function portion of the spec.

23.4. Step 3 — Implement `scan_dependencies`

1. Now implement the main scanning function:

```
@workspace Implement scan_dependencies() in pipeline/
dependency_scanner.py.
Use the specification in openspec/dependency-scanner.spec.md.
The function must:
read requirements lines of the form "package==version" (skip
comments and blank lines),
load the CVE database through data/loader.load_cve_database(),
use is_version_affected()
for version comparison, and return only the matching CVE
records. Remove the
NotImplementedError. Follow copilot-instructions.md.
```

2. Test with the shipping API requirements (which contains the CRITICAL CVE):

```
from pipeline.dependency_scanner import scan_dependencies
hits = scan_dependencies("mock_data/
requirements_shipping_api.txt")
print(f"Found {len(hits)} CVEs")
for h in hits:
    print(f"  {h['cve_id']} ({h['severity']}) — {h['package']}")
# Expect CVE-2024-5678 (CRITICAL) for cryptography 41.0.5
```

3. Test with the clean requirements (should return an empty list):

```
hits = scan_dependencies("mock_data/
requirements_notification_service.txt")
assert hits == [], f"Expected no CVEs, got {hits}"
print("Clean requirements: no CVEs found — correct")
```

4. Run `openspec verify openspec/dependency-scanner.spec.md`.

23.5. Step 4 — Implement `ticket_client.py`

1. Implement `get_blocking_tickets` first:

```
@workspace Implement get_blocking_tickets() in pipeline/
ticket_client.py.
Use openspec/ticket-client.spec.md. The function must filter
tickets by status="open",
blocking_release=True, case-insensitive component match, and
version pattern matching
(exact string or wildcard ending in "x" where "2.4.x" matches
"2.4.0" and "2.4.1").
Load ticket data through data/loader.load_tickets(). Remove the
NotImplementedError.
Follow copilot-instructions.md.
```

2. Test the blocking ticket filter:

```python
from pipeline.ticket_client import get_blocking_tickets
# SHIP-1042 blocks shipping-api 2.4.1
tickets = get_blocking_tickets("shipping-api", "2.4.1")
assert any(t["ticket_id"] == "SHIP-1042" for t in tickets),
"Expected SHIP-1042"
# Notification service 1.2.3 has no blocking tickets
clean = get_blocking_tickets("notification-service", "1.2.3")
assert clean == [], f"Expected no blocking tickets, got {clean}"
print("Ticket client tests pass")
```

3. Implement `get_open_tickets` and run `openspec verify openspec/ticket-client.spec.md`.

23.6. Step 5 — Verify Both Stub Implementations

```
openspec verify openspec/dependency-scanner.spec.md
openspec verify openspec/ticket-client.spec.md
```

Both must pass before Session 3.

23.7. Session 2 Debrief

Before ending Session 2, confirm:

- `is_version_affected("41.0.5", "41.0.6")` returns `True`
- `scan_dependencies("mock_data/requirements_shipping_api.txt")` finds CVE-2024-5678
- `get_blocking_tickets("shipping-api", "2.4.1")` returns SHIP-1042
- `get_blocking_tickets("notification-service", "1.2.3")` returns an empty list
- `openspec verify` passes for both stub specs
- The three functional pipeline modules still run without errors (`python -m pipeline.runner`)

---

24. Challenge 1: Harden the Version Wildcard Matcher

Complete this section if your group finishes the core steps early.

The `get_blocking_tickets` function must handle several edge cases in version matching. Add support for:

- **Major wildcard**: `"3.x"` matches any `3.y.z` version
- **Malformed patterns**: if `affected_versions` contains a string that is neither an exact version nor a valid wildcard pattern, log a warning and skip that record

Write pytest tests in `tests/test_ticket_client.py` covering:

1. `"2.4.x"` matches `"2.4.0"` → True
2. `"2.4.x"` matches `"2.5.0"` → False
3. `"3.0.0"` matches `"3.0.0"` → True
4. `"3.0.0"` matches `"3.0.1"` → False
5. `"3.x"` matches `"3.99.99"` → True
6. `"3.x"` matches `"4.0.0"` → False

---

25. Challenge 2: Add CVE Severity Filtering

Complete this section if your group finishes Challenge 1 early.

Extend `scan_dependencies` with an optional `min_severity` parameter:

```
def scan_dependencies(
    requirements_path: str,
    cve_db_path: str = "mock_data/cve_database.json",
    min_severity: str = "LOW",
) -> list[dict[str, object]]:
    ...
```

Severity levels in ascending order: `LOW < MEDIUM < HIGH < CRITICAL`. When `min_severity="HIGH"`, only return CVE records with severity HIGH or CRITICAL.

Update `openspec/dependency-scanner.spec.md` with the new parameter contract. Run `openspec verify` after the update. Add tests for:

1. `min_severity="CRITICAL"` with label service requirements → only CVE-2024-5678 (CRITICAL) or none
2. `min_severity="LOW"` → all matches returned
3. `min_severity="HIGH"` with notification service requirements → empty list

---

26. Session 3: Build the Models and Wire the Agent

Duration: 3 hours
Copilot mode: Agent Mode (Plan agent for design, Edit mode for implementation)

26.1. Session 3 — Learning Objectives

By the end of Session 3, your group will:

- Have `models.py` implemented and verified
- Have `agent.py` that calls all four checks and produces `ReleaseRecommendation` output

- Be able to run the agent manually against REL-001 (release), REL-003 (hold), and REL-006 (escalate)

26.2. Step 1 — Implement `models.py`

1. In Agent Mode, run:

```
@workspace Implement models.py at the project root using
openspec/models.spec.md.
Define ReleaseRequest and ReleaseRecommendation using Pydantic
v2.
decision must be typed as Literal["release", "hold",
"escalate"].
rationale must be validated as non-empty. Add a docstring to
every class.
Follow copilot-instructions.md.
```

2. Verify:

```
openspec verify openspec/models.spec.md
```

26.3. Step 2 — Design the Agent Architecture

1. Before implementing `agent.py`, run a planning prompt:

```
@workspace I need to plan agent.py for this brownfield project.
The agent must:
accept a ReleaseRequest, call four checks (CVE scan via
dependency_scanner,
blocking tickets via ticket_client, changelog gap via
changelog_reader,
CI gate via deploy), and produce a ReleaseRecommendation. The
agent must NOT
call pipeline/deploy.py directly — it is advisory only. Plan the
system prompt
(including decision priority), tool registration, and error
handling.
Do not write code yet. Output a numbered plan.
```

2. Review the plan with your group. Confirm:
   - The decision priority matches `copilot-instructions.md`: CRITICAL CVE → escalate first
   - The agent calls `ChangelogReader` to detect the major-version gap
   - The agent checks the CI results but does NOT approve or deny deployments
   - Tool errors are reflected in the rationale, not silently swallowed

26.4. Step 3 — Implement Tool Wrappers

1. Create `tools/__init__.py` (empty).
2. Create `tools/release_tools.py` with wrapper functions that connect the agent to the pipeline modules:

```
@workspace Create tools/release_tools.py. Define four functions
that the agent
will register as tools:
(1) scan_release_dependencies(requirements_file: str) — calls
pipeline.dependency_scanner.scan_dependencies
(2) get_release_blocking_tickets(component: str, version: str) —
calls pipeline.ticket_client.get_blocking_tickets
(3) check_changelog_coherence(changelog_path: str, version: str)
— calls
pipeline.changelog_reader.ChangelogReader.detect_major_version_g
ap
(4) evaluate_ci_gate(ci_run_id: str) — loads the CI result from
data/loader.load_ci_results() and calls
pipeline.deploy.DeployGate().evaluate()
Each function must have a complete docstring explaining when the
agent should call it.
Follow copilot-instructions.md.
```

26.5. Step 4 — Implement `agent.py`

Write the OpenSpec for the agent if you haven't already (see Challenge 1, Session 1), then implement it:

```
@workspace Implement agent.py at the project root using openspec/
agent.spec.md.
Register the four tools from tools/release_tools.py. The system
prompt must:
(1) state that the agent is advisory — it never deploys directly
(2) enforce decision priority: CRITICAL CVE → escalate; HIGH CVE +
blocking ticket → escalate;
major version changelog gap → escalate; HIGH CVE only → hold;
blocking ticket only → hold;
CI failures → hold; all clear → release
(3) require that rationale references the specific check(s) that
drove the decision.
Load ANTHROPIC_API_KEY using python-dotenv. Follow copilot-
instructions.md.
```

26.6. Step 5 — Manual Smoke Test

1. Write a temporary test script:

```python
# scratch_test.py — delete before Session 4
import asyncio, json
from pathlib import Path
from agent import agent
from models import ReleaseRequest

requests = json.loads(Path("mock_data/
release_requests.json").read_text())

async def main():
    for req_data in [requests[0], requests[2], requests[5]]:
        # REL-001 (release), REL-003 (hold), REL-006 (escalate)
        req = ReleaseRequest(**{
            k: v for k, v in req_data.items()
            if k not in {"expected_outcome", "outcome_reason"}
        })
        result = await agent.run(str(req))
        print(f"{req_data['request_id']}:
{result.data.decision}")
        print(f"  {result.data.rationale[:80]}...")

asyncio.run(main())
```

2. Confirm the three expected outcomes appear.
3. If the agent produces incorrect decisions, check the system prompt priority language. "Should prefer escalate" is weaker than "must always escalate when."

26.7. Step 6 — Verify Agent Spec

```
openspec verify openspec/agent.spec.md
openspec verify-all
```

26.8. Session 3 Debrief

Before ending Session 3, confirm:

- [ ] `openspec verify-all` passes
- [ ] REL-001 returns `release`, REL-003 returns `hold`, REL-006 returns `escalate`
- [ ] The agent does not import or call `pipeline.deploy` directly
- [ ] Tool errors produce a rationale mentioning the failure (not an unhandled exception)

---

27. Challenge 1: Changelog Coherence Rationale

Complete this section if your group finishes the core steps early.

REL-009 (`notification-service v3.0.0`) should escalate because the changelog has a major version bump with no breaking change documentation. Run REL-009 through your agent and check whether it produces the correct decision.

If the agent produces `release` or `hold` instead of `escalate`, the system prompt may not be clear enough about the changelog gap rule. Refine the system prompt so that a major version bump with a missing "Breaking Changes" section always escalates.

Write a test that asserts REL-009 produces `escalate`.

---

28. Challenge 2: Structured Tool Return Types

Complete this section if your group finishes Challenge 1 early.

The tool wrappers in `tools/release_tools.py` currently return raw dicts or primitive values. Extend `models.py` with structured result types for each tool:

- `CveMatch(cve_id, package, severity, cvss_score, description)`
- `BlockingTicket(ticket_id, title, priority, component)`
- `ChangelogGap(previous_version, current_version, description)`
- `CiGateResult(approved, reason, ci_run_id)`

Update the tool wrappers to return these typed objects. Update `openspec/models.spec.md` with the new types. Run `openspec verify` after the update.

---

29. Session 4: Test and Peer Review

Duration: 3 hours
Copilot mode: Agent Mode (peer review skill invocation, then test generation)

29.1. Session 4 — Learning Objectives

By the end of Session 4, your group will:

- Have a pytest suite covering all four required test cases
- Have run the RAPID ITC.009 peer review Agent Skill
- Have a completed `docs/rapid-peer-review.md` with all findings addressed

29.2. Step 1 — Write the Core Test Suite

1. In Agent Mode, run:

```
@workspace Write a pytest test suite in tests/test_agent.py
covering four cases:
(1) release — use REL-001 (notification-service 1.2.3, clean
requirements, CI passes)
(2) hold (CI failures) — use REL-003 (notification-service
1.1.0, CI-2024-005 fails)
(3) hold (blocking ticket) — use REL-004 (tracking-service
5.2.1, TRACK-088 blocking)
(4) escalate (CRITICAL CVE) — use REL-006 (shipping-api 2.4.1,
cryptography CVE)
Each test must assert result.data.decision equals the expected
outcome and
result.data.rationale is non-empty. Use pytest-asyncio.
```

2. Run:

```
pytest tests/ -v
```

3. Fix all failures. Do not delete tests to make them pass.

29.3. Step 2 — Verify the Existing Pipeline Still Works

1. The functional pipeline modules must not have regressed:

```
python -m pipeline.runner
```

2. If any unexpected behavior appears, check whether your changes to `dependency_scanner.py` or `ticket_client.py` affected how the runner loads data.

> ⓘ Important
>
> If `python -m pipeline.runner` produces different output than in Session 1, that is a regression. The runner only uses `deploy.py` — it does not call the stub implementations. If the output changed, find the cause before Session 5.

29.4. Step 3 — Run the RAPID Peer Review

1. Open a new Agent Mode session.
2. Invoke the peer review Agent Skill:

```
@workspace Run the peer review Agent Skill defined in
.github/skills/rapid-peer-review.md. Perform a full RAPID ITC.
009 code review
of the current implementation. Evaluate all six criteria and
save the findings
to docs/rapid-peer-review.md.
```

3. Read `docs/rapid-peer-review.md` with your group.
4. Pay special attention to Criterion 4 (Reference Architecture Alignment):
   - Did you read the mock data directly in any file other than `data/loader.py`?
   - Did your agent import from `pipeline.deploy` directly?
   - Did any file modify `pipeline/runner.py`, `pipeline/deploy.py`, or `pipeline/changelog_reader.py`?

29.5. Step 4 — Address Findings

1. For each finding rated **Needs Attention** or **Fail**, fix the issue and re-run:

```
pytest tests/ -v && python -m pipeline.runner
```

2. Update the Required Actions section of `docs/rapid-peer-review.md` to note each resolution.

29.6. Step 5 — Full OpenSpec Verify

```
openspec verify-all
```

All specs must pass before Session 5.

29.7. Step 6 — Capture Test Results and Complete RAPID Compliance Artifacts

1. Run the full test suite with results capture (ITC.003):
   ```
   pytest tests/ -v --tb=short --junitxml=docs/test-results.xml
   ```
   `docs/test-results.xml` is a machine-readable record of all test outcomes. All tests must pass before you commit this file.
2. Verify the brownfield pipeline still runs cleanly:
   ```
   python -m pipeline.runner
   ```
3. Open `backoutPlan.md` in VS Code. Fill in the required fields (ITC.013):
   - **Section 1** — run `git log --oneline -1` and paste the hash into "Last stable commit hash"
   - **Section 5** — add your group members' names and contact information
4. Commit all three compliance artifacts together:
   ```
   git add docs/test-results.xml docs/go-no-go-checklist.md
   backoutPlan.md
   git commit -m "[US-005] Add ITC.003 test results and ITC.013
   backout plan"
   ```

29.8. Session 4 Debrief

Before ending Session 4, confirm:

`pytest tests/ -v --tb=short --junitxml=docs/test-results.xml` — all tests pass; `docs/test-results.xml` committed
`python -m pipeline.runner` — produces the same output as Session 1
`docs/rapid-peer-review.md` — all criteria rated and all findings addressed
`openspec verify-all` — passes
`backoutPlan.md` has stable baseline commit hash and contacts filled in

---

30. Challenge 1: Parametrized CVE Severity Tests

Complete this section if your group finishes the core steps early.

Using `@pytest.mark.parametrize`, write tests in `tests/test_dependency_scanner.py` that cover:

- `requirements_notification_service.txt` → empty list
- `requirements_label_service.txt` → at least one HIGH CVE
- `requirements_shipping_api.txt` → at least one CRITICAL CVE
- A manually constructed requirements string with an exact version known to be safe → empty list

Also add parametrized tests for `is_version_affected` covering at least 8 version comparison cases (in-range, at-boundary, above-range, major version difference).

---

31. Challenge 2: Integration Test for All Nine Release Requests

Complete this section if your group finishes Challenge 1 early.

Write a parametrized integration test in `tests/test_integration.py` that:

1. Loads all records from `mock_data/release_requests.json`
2. For each record, constructs a `ReleaseRequest` (excluding `expected_outcome` and `outcome_reason`)
3. Runs the agent
4. Asserts that `result.data.decision == req_data["expected_outcome"]`

Mark the tests with `@pytest.mark.integration` so they can be run separately from unit tests:

```
pytest tests/test_integration.py -v -m integration
```

Note: REL-002 (notification-service 1.3.0) has a missing changelog entry — the agent should handle this gracefully. Document what your agent produces for REL-002.

---

32. Session 5: Full Integration, Go/No-Go, and Showcase

Duration: 3 hours
Copilot mode: Agent Mode for final fixes only

32.1. Session 5 — Learning Objectives

By the end of Session 5, your group will:

- Have run the agent against all nine sample requests
- Have a completed `docs/go-no-go-checklist.md`
- Have presented your agent and key brownfield integration decisions

32.2. Step 1 — Full Integration Run

1. Write a script `run_all_requests.py`:

```
"""Run the agent against all nine sample release requests and
compare to expected."""
import asyncio
import json
from pathlib import Path
from agent import agent
from models import ReleaseRequest


async def main() -> None:
    requests_data = json.loads(
        Path("mock_data/
release_requests.json").read_text(encoding="utf-8")
    )
    results = {"release": 0, "hold": 0, "escalate": 0,
"mismatch": 0}

    for req_data in requests_data:
        expected = req_data["expected_outcome"]
        req = ReleaseRequest(**{
            k: v for k, v in req_data.items()
            if k not in {"expected_outcome", "outcome_reason"}
        })
        result = await agent.run(str(req))
        decision = result.data.decision
        match = "□" if decision == expected else "□"
        results[decision] += 1
        if decision != expected:
            results["mismatch"] += 1
        print(f"{match} {req_data['request_id']}:
expected={expected}, got={decision}")
        print(f"  {result.data.rationale[:80]}...")

    print(f"\nrelease={results['release']}
hold={results['hold']} "
          f"escalate={results['escalate']}
mismatches={results['mismatch']}")


asyncio.run(main())
```

2. Run the script and confirm:
   - At least one `release`, one `hold`, and one `escalate` in the output

- Mismatches are zero for REL-001 through REL-008
- REL-009 (the changelog gap case) produces `escalate`

32.3. Step 2 — Final Acceptance Criteria Check

```
python run_all_requests.py
pytest tests/ -v
python -m pipeline.runner   # Must still match Session 1 output
openspec verify-all
```

Confirm `docs/rapid-peer-review.md` exists and all findings are resolved.

32.4. Step 3 — Complete the Go/No-Go Checklist

1. Open `docs/go-no-go-checklist.md`.
2. Fill in every field:
   - Date: today's date
   - Release description: what your agent does in one sentence
   - Paste pytest summary line into the test results section
   - Paste `openspec verify-all` output
   - Paste the `python -m pipeline.runner` summary (confirm existing pipeline still passes)
   - Peer review rating from `docs/rapid-peer-review.md`
3. Write the Decision Rationale — minimum two sentences. Reference the test results, peer review rating, and acceptance criteria status. Note whether the functional pipeline modules were left intact.
4. Mark the decision checkbox.

32.5. Step 4 — Group Showcase

Each group has **5 minutes**:

Time	Content
0:00–1:00	Demo: run REL-006 (escalate, CRITICAL CVE) live — show the recommendation
1:00–2:30	Brownfield integration: explain what you were NOT allowed to change and why. How did the pre-authored spec guide your design?
2:30–4:00	Show the peer review: open one finding from `docs/rapid-peer-review.md` and explain the resolution
4:00–5:00	Production readiness: what would you add to make this agent safe for a real release pipeline?

32.6. Session 5 Debrief

After all groups present, the instructor will facilitate:

- How does the advisory role of the agent change the design compared to a system that directly gates deployments?
- If a new CVE is published tomorrow, how does the agent pick it up? What would need to change?
- The changelog coherence check required you to read `ChangelogReader` code you did not write. What made that easier or harder?

---

33. Challenge 1: Suppress Low-Severity CVEs from Hold Decisions

Complete this section if your group finishes the core steps early.

The current decision rules treat any CVE ≥ MEDIUM as a hold trigger. In practice, MEDIUM CVEs with a CVSS score below 5.0 may not warrant a hold if no exploit is known and the component is behind a firewall.

1. Add a `suppress_low_risk` flag to the agent system prompt: when True, only CVEs with `cvss_score >= 7.0` trigger a hold or escalate.
2. Update `openspec/agent.spec.md` to document this behavior.

3. Add a test that verifies: with `suppress_low_risk=True`, a release request whose only CVE match is MEDIUM severity with `cvss_score=5.3` still produces `release`.

---

34. Challenge 2: Confidence Score

Complete this section if your group finishes Challenge 1 early.

Add a `confidence: float` field (0.0–1.0) to `ReleaseRecommendation`. The agent's system prompt must instruct the model to assign confidence as follows:

- `1.0`: Single unambiguous CRITICAL CVE — only one possible decision
- `0.8–0.9`: Multiple checks all agree on the same decision
- `0.5–0.7`: Checks present but decision is borderline (e.g., HIGH CVE only, no other flags)
- `< 0.5`: Agent could not determine the decision from available data — must escalate

Update `openspec/models.spec.md`. Run `openspec verify`. Add a test asserting that REL-006 (CRITICAL CVE, unambiguous) produces `confidence >= 0.9`.

---

35. Conclusion

You extended a production brownfield release pipeline with a Pydantic AI agent — reading the existing codebase first, then building only what was missing:

- Read and mapped three functional pipeline modules before writing any new code, establishing the integration boundary the agent must respect
- Specified `ReleaseRequest`, `ReleaseRecommendation`, and all missing components using OpenSpec, keeping the spec one step ahead of the implementation
- Implemented `dependency_scanner` and `ticket_client` stubs to contract, making them available as agent tools without modifying the existing deploy gate

- Wired a Pydantic AI agent that performs all four checks and consistently produces `release`, `hold`, or `escalate` with CVE IDs, ticket IDs, and CI run references in the rationale
- Confirmed the existing pipeline modules continued to pass their tests throughout — the brownfield constraint held from Session 1 to Session 5
- Completed a structured peer review using the RAPID Agent Skill and a Go/No-Go decision gate with written rationale

The brownfield integration discipline you practiced here — read before you write, spec what you own, never break what you inherit — is the pattern that separates reliable AI-native engineering from prototype-quality work.

35.1. Next Steps

- Add a webhook notification step that posts the `ReleaseRecommendation` to a Slack channel or ticketing system before the deploy gate runs
- Extend `scan_release_dependencies` to traverse transitive dependencies, not just the direct requirements file, using `pip-audit` or a similar tool
- Add a `confidence_score` field to `ReleaseRecommendation` and update the system prompt to populate it based on the number and severity of findings
- Replace `mock_data/` with a live Jira or GitHub Issues integration and measure the agent's end-to-end latency on real ticket data

---

36. Appendix A — Quick Reference

36.1. OpenSpec Commands

```
openspec explore          # Analyze domain; summarize existing code
and gaps
openspec propose          # Draft a spec in plain language
openspec specify          # Formalize a proposal into a machine-
verifiable spec
openspec verify <file>    # Check one spec against the
implementation
openspec verify-all       # Check all specs in the openspec/
directory
openspec list             # List all specs and their status
```

36.2. Decision Priority Order

Priority	Decision	When
1 (highest)	`escalate`	CRITICAL CVE; HIGH CVE + blocking ticket; major version changelog gap; CI fail + any CVE
2	`hold`	HIGH CVE only; blocking ticket only; CI test failures
3 (lowest)	`release`	All checks pass — no CVEs, no blocking tickets, CI passes, changelog coherent

36.3. Pipeline Module Summary

Module	Status	Agent interaction
	Functional	

Module	Status	Agent interaction
`pipeline/changelog_reader.py`		Call `ChangelogReader.detect_major_version_gap()` via tool wrapper
`pipeline/deploy.py`	Functional	Call `DeployGate().evaluate()` via tool wrapper — do NOT import in agent.py directly
`pipeline/runner.py`	Functional	Not called by agent — runs independently as the deploy orchestrator
`pipeline/dependency_scanner.py`	Implemented by you	Call `scan_dependencies()` via tool wrapper
`pipeline/ticket_client.py`	Implemented by you	Call `get_blocking_tickets()` via tool wrapper

36.4. Acceptance Criteria Checklist

#	Criterion
1	Agent accepts `ReleaseRequest`, returns `ReleaseRecommendation`
2	Decision is always `release`, `hold`, or `escalate`
3	Every recommendation includes a non-empty `rationale`
4	All four checks performed: CVE scan, tickets, changelog, CI gate
5	Tool errors are caught and reflected in output
6	All three decision types are reachable with sample requests
7	Functional pipeline modules continue to pass their tests
8	pytest suite passes: release, hold (CI), hold (ticket), escalate (CRITICAL CVE)

#	Criterion
9	`openspec verify-all` passes
10	`docs/rapid-peer-review.md` exists and all findings addressed
11	Agent does not directly invoke `pipeline/deploy.py`